

ESG RAG Pipeline

Technical Design, Implementation Choices & Limitations

Mansi Saxena

June 15, 2026

1 Overview

This pipeline extracts structured ESG data from four steel company sustainability reports using Retrieval-Augmented Generation. Three constraints shaped every design decision: auditability (every answer must carry a source file and page number), numerical accuracy (ESG emissions data is precise and unit-sensitive), and multi-model comparability (the same retrieval backbone is used across all models so that generation quality is the isolated variable).

The companies covered are JFE Holdings (Japan), JSW Steel (India), POSCO (South Korea), and Tata Steel UK. Each report has different length, language mix, fiscal year convention, unit system, and table structure, which the pipeline must handle correctly.

2 PDF Loading

2.1 Library Choice: *pdfplumber*

pdfplumber was chosen over the two most common alternatives, *pypdf* and *PyMuPDF*. The deciding factor was spatial awareness: *pdfplumber* uses the bounding boxes of words on the page to infer table structure and return row and column data directly. This is essential for sustainability reports, where the most important data sits in tables. *pypdf*'s table extraction capabilities are more limited, making it less reliable for structured data extraction. *PyMuPDF* provides similar spatial capabilities but is distributed under the AGPL licence, which may conflict with commercial deployment contexts.

pdfplumber is itself a wrapper around *pdfminer.six*, a lower-level library. The abstraction is worth the trade-off: *pdfplumber*'s ergonomics are substantially cleaner than working with *pdfminer.six* directly.

2.2 Known Limitation: Image-only Pages

Text extraction returns an empty string for scanned or image-only pages, because there is no text layer to process. The four reports in this assignment all have text layers embedded, so this was not an issue here. In a production system processing arbitrary sustainability reports, the correct mitigation is OCR: rasterise each page, then run an OCR engine on pages that return empty strings.

3 Chunking Strategy

3.1 Why Table-Aware Chunking is Crucial

The dominant failure mode in the first version of this pipeline was not retrieval quality or model capability: it was that sliding-window chunking was splitting table rows in half. A chunk boundary falling mid-row means one chunk contains a year label and the first few cells, while the next chunk contains the remaining cells with no year context. The number *70,588,012* appearing without its row header *Scope 1 Direct Emissions* and column header *2023* is unrecoverable by any downstream model.

The fix is to extract tables first, treating each table as an atomic, indivisible unit. Loading and chunking are therefore merged into a single integrated pass rather than sequential steps, because table extraction requires the live page object and cannot operate on pre-extracted text.

3.2 Sliding Window on Remaining Prose

After tables are extracted as atomic chunks, the remaining prose from each page is cleaned (table cell content is removed to avoid double-indexing) and split with a character-based sliding window: 512 characters per chunk with 64-character overlap. The 64-character overlap ensures a number at a chunk boundary appears in full in at least one chunk.

3.3 Within-Page Constraint

Chunks never cross page boundaries. This guarantees that every chunk has exactly one page number as its source, which is a structural requirement for auditable ESG citations.

3.4 Alternatives Considered

Sentence-based chunking was rejected because PDF text extraction does not reliably preserve sentence boundaries; mid-sentence line breaks, hyphenated words, and navigation headers mixed into body text all break sentence tokenisers. Subsection-based chunking was rejected because *pdfplumber* returns a text flow, not a structured subsection hierarchy. Semantic chunking, which splits on embedding similarity, would require encoding the document twice and was not justified given the corpus size.

The production approach used by LlamaIndex and similar frameworks is a hybrid: coarse structure-aware splits on section headings, with fixed-size sub-chunking within each section. This keeps semantically coherent sections together but requires reliable heading detection, which is non-trivial in PDF extraction.

4 Embedding Model and Vector Index

4.1 Embedding Model: all-MiniLM-L6-v2

all-MiniLM-L6-v2 is a sentence-transformers model with 22 million parameters producing 384-dimensional embeddings. It was chosen primarily for speed: the full corpus embeds in under two minutes on CPU, which is acceptable given that embedding is a one-time cost per corpus. More importantly, all LLMs in this pipeline are evaluated against the same embedding backbone. Using a shared retrieval layer isolates generation quality as the variable of interest.

A significant limitation is that **all-MiniLM-L6-v2** is trained predominantly on English text. POSCO's report contains Korean-language sections, and embedding quality on those sections is lower. The correct production fix is **paraphrase-multilingual-MiniLM-L12-v2**, which was trained on parallel data in over 50 languages.

4.2 FAISS IndexFlatIP: Exact Search

The vector index uses FAISS IndexFlatIP, which performs exact nearest-neighbour search via inner product (equivalent to cosine similarity on L2-normalised vectors). At the scale of this corpus, exact search is fast and introduces no recall loss. Approximate indices such as IndexIVFFlat or IndexHNSW trade retrieval accuracy for speed; for a corpus of this size, that trade-off is not justified — the speed gain is negligible and the recall loss is unnecessary.

4.3 Stratified Per-Company Retrieval

A critical design choice is building per-company sub-indices and querying each independently. The JFE Holdings report is substantially longer than the others; a single query against the full index would allocate most retrieval slots to JFE simply because it has more chunks, potentially leaving JSW Steel, POSCO, and Tata Steel with no representation. The stratified approach guarantees exactly top-*k*

chunks from each company regardless of corpus imbalance. Sub-indices are sliced from the already-computed embedding matrix at build time, so no re-encoding is needed.

5 Hybrid Retrieval: FAISS + BM25 + Cross-Encoder

5.1 The Failure Mode of Pure Semantic Search for Numbers

FAISS retrieval operates on semantic similarity. A query *Scope 1 emissions 2024* is encoded into a vector, and chunks with nearby vectors are returned. The problem is that the most relevant chunk for numerical extraction is not necessarily the most semantically similar one. A chunk describing Scope 1 methodology may embed very close to the query, while the chunk containing the actual figures may embed further away because it consists primarily of numbers and abbreviated column headers.

This is not a failure of the embedding model; it reflects a genuine property of numerical data. Numbers have low semantic entropy in embedding space. The string *70,588,012 tCO₂e* has almost no meaningful vector relationship to *Scope 1 emissions 2023* because numbers do not carry conceptual meaning the way words do.

5.2 BM25: Keyword Retrieval

BM25 (Okapi BM25) ranks documents based on exact term frequency and document length normalisation. Unlike embedding-based retrieval, BM25 scores exact token matches between the query and each chunk. For the queries in this pipeline, BM25 surfaces chunks containing the exact vocabulary from the question: terms such as *Scope 1*, *Scope 3*, *CCUS*, *carbon capture*, *net-zero*, and specific years. FAISS might miss a chunk that uses this terminology alongside dense numerical content, because the chunk’s overall embedding is pulled toward methodology text rather than data tables; BM25 surfaces it regardless, because the tokens match.

The pipeline runs FAISS and BM25 independently, deduplicates the combined candidate set, and passes all candidates to the reranker. This two-stage approach is standard in production retrieval systems, often called hybrid retrieval or sparse-dense fusion. Microsoft Azure AI Search, Elastic’s ELSER, and Cohere’s Rerank API all implement variations of this pattern.

5.3 Cross-Encoder Reranking

FAISS and BM25 both score the query and each chunk independently. A cross-encoder, by contrast, takes the query and a candidate chunk as a single concatenated input. Because query tokens attend directly to chunk tokens through the transformer’s self-attention mechanism, the cross-encoder can capture subtle interactions that independent scoring cannot — for example, that *Scope 1* in the query refers to the direct emissions row in a table, not the adjacent indirect emissions row.

The trade-off is that cross-encoders cannot be pre-computed; they must be run fresh for every query–chunk pair. For a candidate set of 80 chunks, this adds approximately 15 seconds on CPU, which is acceptable because the GPU is occupied with LLM generation and the reranker runs concurrently. The reranker also trims the candidate set from 80 to 40 chunks, reducing hallucination pressure on the LLM.

5.4 SOTA Context

Hybrid retrieval with reranking is the current state of the art in production RAG systems. The two-stage pattern — broad candidate retrieval followed by accurate cross-encoder reranking — was established by Nogueira & Cho (2019) with BERT-based passage reranking and is now standard practice. An alternative gaining traction is Reciprocal Rank Fusion (RRF), which fuses ranked lists from multiple retrievers without a separate reranking model. RRF is simpler and faster but less accurate than cross-encoder reranking for domain-specific queries.

6 Q1 Sub-query Decomposition

6.1 Why a Single Query Fails for Q1

Question 1 asks for Scope 1, 2, and 3 emissions and total energy consumption for three years across four companies: 48 data points in a single query. Even with stratified retrieval guaranteeing 10 chunks per company, the LLM must parse four different table formats, four fiscal year conventions, and four unit systems simultaneously in a 40-chunk context window. Observed failure modes include: reading the wrong row from a multi-row-group table (JFE), assigning values to the wrong year due to ambiguous column headers (Tata Steel UK), and conflating Scope 1 and Scope 2 from adjacent rows (POSCO).

6.2 Per-Company Sub-queries

The solution is to decompose Q1 into four independent sub-queries, one per company. Each sub-query receives a focused context containing only that company’s chunks. The LLM has one table format to interpret, one fiscal year convention, and one unit system. A minimal reporting hint is appended to each sub-query system prompt containing metadata about the reporting convention, specifically fiscal year alignment and units.

6.3 Why Q2 and Q3 Do Not Need Sub-queries

Q2 and Q3 are qualitative questions that can be answered from a single multi-company context. CCUS technology descriptions and net-zero targets are expressed in natural language sentences, not in multi-row-group tables with ambiguous column headers. All models identify all four CCUS companies (Q2) from a single-query context. Moreover, sub-querying would incur four times the latency with minimal performance benefit for these questions.

7 LLM Inference

7.1 Framework: vLLM

vLLM is used for all model inference. Its key contribution is PagedAttention, a memory management technique that allocates KV cache memory in non-contiguous fixed-size blocks, enabling higher throughput and better memory utilisation than standard implementations. All models are loaded in offline mode, preventing any network access at inference time.

7.2 Sequential Model Loading

Models are loaded one at a time on a single H100 80GB SXM5 GPU: each model is initialised, all three questions are run, and then the model is explicitly unloaded and the GPU cache cleared before the next model loads. The sequential approach means wall time scales linearly with model count, but it keeps the memory budget predictable and the implementation simple.

7.3 4-bit Quantisation

Three models exceed the H100’s 80GB capacity in native precision: Llama 3.1 70B (~140GB in bfloat16), Qwen3 32B (~64GB), and Mistral Large 123B (~246GB). These are loaded in 4-bit quantisation, reducing memory by approximately four times at the cost of some numerical precision. Quantisation is recorded in output metadata so results can be interpreted accordingly.

7.4 Prompt Design: Preventing Per-Chunk Mimicry

A systematic failure was observed in early runs, particularly with Mistral 7B: when the context contained 40 numbered chunks, the model produced 40 separate responses, one per chunk. This arises because the numbered list format resembles instruction-following templates in the model’s training data, where numbered inputs often expect numbered outputs.

The fix is explicit instruction in the system prompt stating that the numbered excerpts are source material to be synthesised, not items to be responded to individually. This framing makes the role of the numbered context unambiguous.

7.5 Temperature: 0.1

A temperature of 0.1 produces near-deterministic output. High temperature introduces sampling variance into number extraction, which is undesirable for precise numerical ESG data. Low temperature aligns with the requirement for accurate extraction rather than creative generation.

8 Agent Routing

8.1 Rule-Based Router

The pipeline implements a lightweight rule-based query router that classifies queries into four actions: retrieve from PDFs, web search, structured extraction, or request clarification. The router uses keyword matching against known company names, live-data signals, ambiguity signals, and structured-output signals. This is deterministic and zero-cost.

The limitation is brittleness: it misses implicit references (“the Japanese steelmaker”), abbreviations, and paraphrases. “What has Nippon Steel’s rival in Osaka done about carbon?” would not route to POSCO despite being answerable from the corpus.

8.2 Production Alternatives

The most robust alternative is an LLM-based tool selection, where the router itself is a small language model that receives the query and a list of available tools with descriptions, and returns a structured routing decision. This handles paraphrases and implicit references but adds latency and cost per query.

A conceptually different approach that avoids routing entirely is query decomposition: rewrite each incoming question into company-specific sub-queries and run all of them, then merge results. This guarantees full coverage regardless of how a question is phrased. Related strategies in this direction include HyDE [1], which generates a hypothetical answer document and retrieves on its embedding; LlamaIndex’s MultiStepQueryEngine, which decomposes complex queries into sub-steps; and FLARE [2], which iteratively retrieves based on predicted future context. The cost is multiple retrieval and generation calls per question, which may be acceptable where recall matters more than latency.

9 Observed Failure Modes

9.1 JFE Holdings: Multi-Row-Group Table Structure

JFE’s Scope 1 and 2 emissions table on PDF page 259 contains three row groups: JFE Steel Group (all subsidiaries), JFE Steel (standalone), and individual subsidiaries. All models retrieved this page correctly but consistently extracted values from the wrong row; some reported the standalone figure, others the group total; none reliably extracted the “All group” row corresponding to ground truth.

This is a table parsing problem, not a retrieval problem. The correct chunk is retrieved, but the TSV representation of the table does not make the row group hierarchy explicit enough for the LLM to navigate reliably. The correct fix is structured table representation: pre-annotate each row with its group label before indexing, so the model can unambiguously identify the relevant row.

9.2 JSW Steel: Indian Lakh-Crore Number Notation

JSW Steel reports in Indian lakh-crore notation. In this system, commas are placed after every two digits from the right (except the last three): 53,100,751 is written as 5,31,00,751. Most LLMs trained predominantly on English-language text do not recognise this notation and either report it verbatim or misinterpret it as a Western thousands-separated number, producing values off by one or two orders of magnitude. Larger models in this evaluation handle the notation correctly; smaller models do not.

9.3 Tata Steel UK: Fiscal Year Column Assignment

Tata’s data table on PDF page 52 presents five columns: FY21, FY22, FY23, FY24, FY25. When the table header row is not included in the retrieved chunk (which occurs if the chunk boundary falls between the header and data rows), the model assigns values to years by position rather than label. This produces a systematic off-by-one error: FY22 values are reported as FY23, FY23 as FY24, and so on. The error is perfectly consistent across all affected models, confirming it is a chunking artefact rather than a model reasoning failure.

9.4 POSCO: Scope 1 and Scope 2 Conflation

POSCO presents Scope 1 (Direct emissions) and Scope 2 (Indirect emissions) in adjacent rows of the same table. Several models combined them into a single Scope 1+2 total. The ground truth values are Scope 1 = 70,588,012 tCO₂e and Scope 2 = 1,383,895 tCO₂e for 2023; a model that sums them reports approximately 71.97 million tCO₂e, a plausible-looking Scope 1 figure that is hard to detect as incorrect without reference values.

9.5 Q3: Retrieval Failure for 2050 Targets

Both JSW Steel (“Net Neutral by 2050”) and POSCO (“Carbon Neutral by 2050”) have clearly stated net-zero targets in their reports, but these were not consistently surfaced for Q3. The cause is a retrieval failure: the 2050 commitment language appears on pages that rank lower than 2030 interim target pages for the Q3 query vector, because the 2030 language co-occurs with emissions data that the retriever associates with numerical extraction queries. A targeted fix would be to augment the Q3 query with commitment-specific terminology, or to run Q3 as per-company sub-queries.

10 Model Performance Summary

Table 1: Model performance across three evaluation questions

Model	Size	Q1 Numerical	Q2 CCUS	Q3 Net-Zero
Mistral 7B	7B fp16	Poor — per-chunk format	3/4	2/4
Llama 3.1 8B	8B fp16	Partial — JSW lakh errors	2/4	2/4
Qwen3 14B	14B fp16	Partial — JFE broken	4/4	2/4
Gemma 3 27B	27B bf16	Best small — JSW correct	4/4	2/4
Llama 3.1 70B	70B 4-bit	Moderate — Tata empty	4/4	2/4
Qwen3 32B	32B 4-bit	Moderate — S2 10× errors	4/4	2/4
Mistral Large	123B 4-bit	Poor — derives S1 by subtraction	4/4	2/4

* Mistral 7B, Llama 8B, and Qwen 14B scores are from an earlier pipeline version (before table-aware chunking, BM25, and cross-encoder reranking). The final pipeline was run on the four large models only.

Model size does not consistently predict Q1 numerical accuracy. Mistral Large 123B, the largest model, performs worst on Q1 by incorrectly deriving Scope 1 via arithmetic from a misidentified total, producing plausible-looking but incorrect values. Gemma 27B correctly extracts all JSW Steel scope values in lakh notation, outperforming larger models on that company. The primary determinant of Q1 accuracy is whether the retrieved chunk contains a cleanly structured table with unambiguous row labels, not model size.

Q2 is effectively solved at all model sizes above 14B. All models from Qwen 14B upward correctly identify all four companies as having CCUS activity.

Q3 results are consistent across all four large models at 2/4. Llama 70B and Mistral Large correctly retrieve JFE Holdings’ 2050 target; Gemma 27B correctly retrieves JSW Steel’s 2050 target; all models correctly extract Tata Steel UK’s 2045 target. No model retrieves POSCO’s 2050 target. The variation in which company is missed reflects which pages each model’s retrieval happened to surface, rather than a consistent capability difference.

11 Potential for Improvements

Persistent index: The pipeline rebuilds retrieval indices from scratch each run, taking approximately two minutes. In production, indices should be serialised to disk and reloaded on subsequent runs, rebuilding only when the corpus changes.

Multilingual embeddings: Replacing the current embedding model with a multilingual variant (such as `paraphrase-multilingual-MiniLM-L12-v2`) would improve retrieval quality for POSCO’s Korean-language sections.

Structured output enforcement: For Q1, constraining the LLM to emit JSON matching a pre-defined schema would eliminate post-hoc output parsing and reduce year-assignment errors by forcing the model to populate named fields. vLLM supports grammar-constrained decoding for this purpose.

Page-neighbourhood retrieval: JFE’s emissions data spans multiple adjacent pages. Global reranking may not select all relevant pages together. Including adjacent pages automatically when any page from a known data section is retrieved would improve completeness for multi-page tables.

References

- [1] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels, December 2022.
- [2] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation, May 2023.